

IOCL GRADE-A 2026

CS/IT WORKBOOK

DAY 4 — TRANSACTIONS & SERIALIZABILITY

22 AUGUST 2026 (Corrected Edition)

Day 4 Objective

Day 4 moves from DBMS design/normalization into transaction processing: transactions, ACID properties, transaction states, schedules, serial vs. non-serial schedules, conflict serializability, precedence graphs, and recoverability. This keeps the same pass-oriented approach: learn standard objective-question patterns and small calculations rather than pursuing advanced database theory.

Recommended Day 4 Workload

Block	Time	What to do
Concepts	50–55 min	Transactions, ACID, states, schedules, notation
Serializability	30–35 min	Conflicts, precedence graph, conflict serializability
Recovery	20–25 min	Recoverable, cascadeless, strict schedules
MCQs	25–30 min	Guided + independent objective questions
Timed mini-test	18 min	12 mixed questions
Error log	10 min	Record traps and revise
Cumulative review	10 min	Rapid-fire Days 1–3 (no reteaching)
Daily quiz	5 min	5 core concept check

Total: approximately 2 hours 45 minutes.

Scope Boundary

Do cover: transaction basics, ACID, transaction states, schedules, serial/non-serial schedules, conflicting operations, conflict serializability, precedence graphs, schedule notation, view-serializability at recognition level, recoverable/cascadeless/strict schedules, and common MCQ traps.

Do not cover deeply: two-phase locking, deadlock handling, timestamp protocols, MVCC implementation details, distributed transactions, recovery algorithms such as ARIES, or advanced view-serializability proofs.

Contents

1	Learning Objectives	3
2	Concepts and Mental Models	3
2.1	A. Transaction Basics	3
2.2	B. ACID Properties	4
2.3	C. Transaction States	4
2.4	D. Schedules and Schedule Notation	5
2.5	E. Conflicting Operations	6
2.6	F. Conflict Serializability and Precedence Graphs	6
2.7	G. View Serializability (Recognition Level)	7
2.8	H. Recoverability: Recoverable, Cascadeless, Strict Schedules	8
2.9	I. Common Traps	9
3	Guided Practice (10 Questions)	11
3.1	Transaction Basics & ACID (2 questions)	11
3.2	Schedules & Conflicts (2 questions)	11
3.3	Conflict Serializability (3 questions)	11
3.4	Recoverability (3 questions)	11
4	Guided Practice — Full Worked Solutions	13
5	Independent Practice (18 Questions)	15
5.1	ACID & Transaction States (4 questions)	15
5.2	Schedules & Conflicts (4 questions)	16
5.3	Conflict Serializability (5 questions)	16
5.4	Recoverability (5 questions)	17
6	Timed Mini-Test (12 Questions — 18 minutes)	19
7	Day 1–3 Cumulative Review (8 Rapid-Fire Questions)	21
8	Daily Quiz (5 Questions)	23
9	Complete Answer Key	24
9.1	Independent Practice Answers	24
9.2	Timed Mini-Test Answers	24
9.3	Cumulative Review Answers	25
9.4	Daily Quiz Answers	25
10	Scoring and Diagnostic Interpretation	26
11	Revision Checklist	27
12	Things I Got Wrong Today	27
13	One-Page Cheat Sheet	28

1 Learning Objectives

By the end of Day 4, you should be able to:

- Define a database transaction and explain why transactions exist.
- Identify the four ACID properties and their practical meanings.
- Identify transaction states and state transitions.
- Distinguish serial and non-serial schedules.
- Understand read/write operations and their notation: $r_i(X)$, $w_i(X)$, c_i , a_i .
- Identify conflicting operations using the exact rule.
- Construct a precedence/serialization graph for a small schedule.
- Determine whether a schedule is conflict-serializable.
- Derive a possible serial order from an acyclic precedence graph.
- Recognize the basic idea of view serializability without doing advanced proofs.
- Distinguish recoverable, cascadeless, and strict schedules.
- Understand the critical distinction: serializability and recoverability are **separate** correctness properties.
- Identify common schedule/serializability traps quickly.

2 Concepts and Mental Models

2.1 A. Transaction Basics

What: A transaction is a logical unit of work that reads and/or writes to the database and must move the database from one consistent state to another. A transaction has a clear beginning and end.

Why it exists: Without transactions, a system crash mid-operation could leave the database in an inconsistent state. Transactions group related operations so the database either completes all of them (commit) or none of them (abort/rollback).

Example: Bank transfer from account A to B. A transaction might read A's balance, subtract 100, write A's new balance, read B's balance, add 100, write B's new balance. If a crash occurs between writing A and writing B, the transfer is incomplete and money is lost. A transaction ensures either both writes succeed or both are undone.

Mental Model: "One logical unit of work that should move the database from one valid state to another."

Commit and Rollback

- **Commit:** The transaction has completed successfully; all its changes are permanently written to the database (made durable).
- **Rollback (Abort):** The transaction encountered a failure or conflict; all its changes are undone, and the database is restored to the state before the transaction began.

2.2 B. ACID Properties

ACID is an acronym for four properties that a database system should guarantee for transactions:

Property	Meaning	Practical Example	Common MCQ Wording
Atomicity	All-or-nothing: a transaction completes fully or not at all.	Bank transfer: either both writes happen or neither.	“Transaction partially executes and fails.”
Consistency	The database moves from one valid state to another; business rules and integrity constraints are preserved.	Bank transfer preserves the total money in the system (no creation or destruction).	“Invariant or constraint is violated after a transaction.”
Isolation	Concurrent transactions do not interfere with each other; the result is as if they ran sequentially.	Two customers withdrawing from the same account simultaneously should not both see the old balance and both deduct.	“Dirty read, lost update, non-repeatable read.”
Durability	Once a transaction commits, its changes survive any system failure (crash, power loss, etc.).	After a withdrawal commits, even a power loss won't reverse it.	“Successful commit survives crash.”

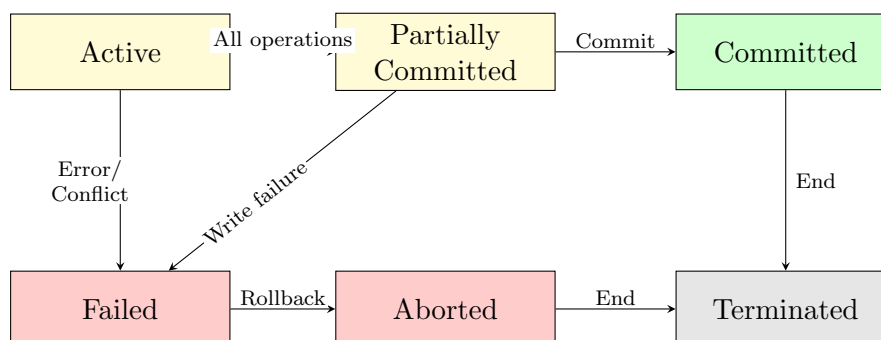
Common Trap

Trap 1: Do not confuse database consistency (ACID C) with isolation (ACID I). Consistency means business-rule preservation; isolation means concurrent transactions don't see each other's uncommitted work.

Trap 2: ACID guarantees do not automatically ensure application-level correctness—they guarantee database-level integrity. A broken application might still transfer money to the wrong account.

2.3 C. Transaction States

A transaction passes through several states during its lifetime:



State Definitions:

- **Active:** The transaction is currently executing; read/write operations are ongoing.
- **Partially Committed:** All operations have been executed successfully, but changes have not yet been written to stable storage (durability not guaranteed yet).
- **Committed:** All changes have been written to stable storage; the transaction has succeeded and is now permanent.
- **Failed:** The transaction encountered an error (constraint violation, system failure, etc.) and cannot continue.
- **Aborted:** The failed transaction has been rolled back; all its changes are undone, and the database is restored to the pre-transaction state.
- **Terminated:** The transaction has either committed successfully or been fully aborted and is no longer active.

2.4 D. Schedules and Schedule Notation

What: A schedule is a temporal ordering of the read and write operations of multiple concurrent transactions.

Schedule Notation (Critical for Exam):

- $r_i(X)$ — Transaction T_i reads data item X.
- $w_i(X)$ — Transaction T_i writes (modifies) data item X.
- c_i — Transaction T_i commits.
- a_i — Transaction T_i aborts.

Example Schedule: For transactions T_1 and T_2 :

$$r_1(X) \rightarrow w_1(X) \rightarrow r_2(X) \rightarrow w_2(X) \rightarrow c_1 \rightarrow c_2$$

This means: T_1 reads X, T_1 writes X, T_2 reads X, T_2 writes X, T_1 commits, T_2 commits.

Serial vs. Non-Serial:

- **Serial Schedule:** Transactions execute one after another with no interleaving. Example: T_1 completes entirely, then T_2 starts.
- **Non-Serial Schedule:** Operations of different transactions are interleaved.

Why Interleaving: Interleaving allows concurrency, improving throughput and responsiveness. However, interleaving can cause anomalies (lost updates, dirty reads, etc.).

Common Trap

Critical Distinction: A schedule being **serial** (no interleaving) is not the same as being **serializable** (equivalent to some serial schedule). A non-serial schedule can be serializable if its result is equivalent to at least one serial execution. This is the entire point of serializability: allowing interleaving while guaranteeing correctness.

2.5 E. Conflicting Operations

Definition: Two operations conflict if:

1. They belong to **different** transactions.
2. They access the **same** data item.
3. At least one is a **write**.

Conflict Table:

	$r_j(X)$	$w_j(X)$
$r_i(X)$	No	Yes
$w_i(X)$	Yes	Yes

Key Insight: $r_i(X)$ and $r_j(X)$ do **not** conflict—multiple transactions can read the same item simultaneously without issue.

Worked Example: In the schedule $r_1(X) \rightarrow w_2(X) \rightarrow r_1(Y)$:

- $r_1(X)$ and $w_2(X)$ **conflict** (different transactions, same item X, write present).
- $w_2(X)$ and $r_1(Y)$ do **not** conflict (different items).

Common Trap

The most common trap: assuming $r_i(X)$ and $r_j(X)$ conflict. **They do not.**

2.6 F. Conflict Serializability and Precedence Graphs

Definition: A schedule is **conflict-serializable** if its operations can be reordered (without violating the order of operations within each transaction) so that conflicting operations of different transactions are never interleaved—i.e., the result is equivalent to some serial schedule.

Standard Exam Method: Precedence Graph

A precedence graph (also called serialization graph) is a directed graph where:

- **Nodes** = transactions.
- **Edges** = ordering constraints from conflicts.

When to Add an Edge: For every pair of conflicting operations o_i (in transaction T_i) and o_j (in transaction T_j) where o_i occurs before o_j in the schedule, add edge $T_i \rightarrow T_j$ to the graph.

Serializability Test:

- **If the graph is acyclic:** The schedule is conflict-serializable. A topological sort of the graph gives a valid serial order.
- **If the graph has a cycle:** The schedule is **not** conflict-serializable.

Worked Example: Conflict Serializability

Schedule: $r_1(X) \rightarrow w_2(X) \rightarrow r_2(Y) \rightarrow w_1(Y) \rightarrow c_1 \rightarrow c_2$

Step 1: Identify conflicts

- $r_1(X)$ and $w_2(X)$ conflict: $T_1 \rightarrow T_2$.
- $w_1(Y)$ and $r_2(Y)$ conflict: but $r_2(Y)$ occurs before $w_1(Y)$ in the schedule, so edge is $T_2 \rightarrow T_1$.

Step 2: Draw the graph

$$T_1 \rightarrow T_2 \rightarrow T_1$$

This is a **cycle** ($T_1 \rightarrow T_2 \rightarrow T_1$), so the schedule is **not conflict-serializable**.

Exam Shortcut Box:**To build a precedence graph quickly:**

1. Scan the schedule left to right.
2. For each pair of operations: if they conflict and the first is in T_i , the second in T_j , draw $T_i \rightarrow T_j$.
3. If all edges form a DAG (directed acyclic graph), the schedule is conflict-serializable.
4. Topological sort gives a valid serial order.

Common Trap

Trap 1: Reversing the edge direction. The edge goes from the transaction with the earlier conflicting operation.

Trap 2: Thinking a cycle means “not serializable.” A cycle means “not conflict-serializable,” but the schedule might still be view-serializable (see Section F.2).

2.7 G. View Serializability (Recognition Level)

Definition (Awareness Only): Two schedules are view-equivalent if they have the same reads-from relationships (which writes each read sees) and the same final writes.

A schedule is **view-serializable** if it is view-equivalent to some serial schedule.

Key Fact: Every conflict-serializable schedule is view-serializable, but the reverse is not always true. There exist schedules that are view-serializable but not conflict-serializable.

Tiny Example: View-Serializable but Not Conflict-Serializable

Schedule: $w_1(X) \rightarrow w_2(X) \rightarrow w_1(X) \rightarrow c_1 \rightarrow c_2$

Conflict-serializability check: There are no reads here, only writes on the same item X, so every pair of operations on X conflicts (write-write).

- $w_1(X)$ (first occurrence) before $w_2(X)$: edge $T_1 \rightarrow T_2$.
- $w_2(X)$ before $w_1(X)$ (second occurrence): edge $T_2 \rightarrow T_1$.

This gives the cycle $T_1 \rightarrow T_2 \rightarrow T_1$, so the schedule is **not conflict-serializable**.

View-serializability check: There are no reads at all, so the reads-from relationship is trivially empty and imposes no constraint. The only thing that matters is the **final write** on X, which is by T_1 (the last write in the schedule). In the serial order (T_2, T_1) , T_2 writes X first and T_1 writes X last—so T_1 is again the final writer. The final-write condition matches, so this schedule is **view-serializable** (view-equivalent to the serial order $T_2 \rightarrow T_1$), even though it is not conflict-serializable.

Key lesson: Conflict serializability is a sufficient but not necessary condition for view serializability. For CBT purposes, conflict-serializable is the standard default; view-serializable-but-not-conflict-serializable cases are rare and only need recognition, not construction.

2.8 H. Recoverability: Recoverable, Cascadeless, Strict Schedules

Core Problem: If a transaction T_j reads data written by T_i , and T_i later aborts, should T_j be rolled back too?

Recoverable Schedule

Definition: A schedule is **recoverable** if, whenever T_j reads a value written by T_i , the commit of T_i occurs before the commit of T_j .

In other words: “Don’t commit a transaction that has read uncommitted data until the writer has committed.”

Worked Example:

- Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow c_2 \rightarrow c_1$. T_2 reads X (written by T_1), but T_2 commits before T_1 commits. **Not recoverable.** If T_1 later aborts, we cannot undo T_2 ’s changes (it already committed based on T_1 ’s uncommitted write).
- Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow c_1 \rightarrow c_2$. T_2 reads X (written by T_1), and T_1 (the writer) commits before T_2 commits. **Recoverable.** If T_1 were going to abort, it’s resolved before T_2 commits.

Cascadeless Schedule

Definition: A schedule is **cascadeless** if transactions only read committed data. Formally, T_j may not read data written by T_i unless T_i has already committed.

Why it matters: Cascadeless schedules prevent cascading rollbacks. If T_i aborts, only T_i needs to roll back—not all the transactions that read from it.

Recoverable but not Cascadeless:

- Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow c_1 \rightarrow c_2$. T_2 reads X before T_1 commits (not cascadeless), but T_1 commits before T_2 commits (recoverable). If T_1 had aborted after T_2 read but before T_1 committed, T_2 would need to be rolled back (cascading rollback).

Strict Schedule

Definition: A schedule is **strict** if, whenever T_i writes X, no **other** transaction T_j may read or write X until T_i commits or aborts.

Note the emphasis on **other**: a transaction is always free to read or write its own uncommitted data (e.g., T_1 writing X and then later reading its own value of X before committing is perfectly normal). Strictness only restricts **other** transactions from touching an item that some uncommitted transaction has written.

Why it matters: Strict schedules simplify recovery. After a crash, you only need to undo the writes of uncommitted transactions; recovery does not need to replay committed transactions.

Hierarchy:

$$\text{Strict} \implies \text{Cascadeless} \implies \text{Recoverable}$$

The reverse implications do **not** hold in general.

Common Trap

Critical Distinction: Serializability (Section F) and recoverability (this section) are **completely separate** correctness properties. A schedule can be:

- Serializable but non-recoverable.
- Recoverable but non-serializable.
- Both serializable and recoverable (ideal).
- Neither (problematic).

This is one of the most important conceptual distinctions for Day 4 and frequently tested in PSU/GATE papers.

2.9 I. Common Traps

Explicitly covered traps (review these daily):

1. $r_i(X)$ and $r_j(X)$ **conflict: False**. Multiple reads of the same item do not conflict.
2. **Reversing precedence-graph edge direction:** Always draw $T_i \rightarrow T_j$ when T_i 's conflicting operation occurs **before** T_j 's.
3. **Cycle in precedence graph $\implies$ serializable: False**. A cycle means **not conflict-serializable**.
4. **Confusing serial with serializable:** Serial means no interleaving. Serializable means equivalent to some serial execution (interleaving is OK).
5. **Confusing conflict-serializable with view-serializable:** Every conflict-serializable is view-serializable, but not vice versa.
6. **Serializability $\implies$ Recoverability: False**. They are independent properties.
7. **Recoverability $\implies$ Serializability: False**. A schedule can be recoverable but non-serializable.
8. **Confusing recoverable with cascadeless:** Recoverable is weaker; cascadeless is stricter.

9. **Strict only blocks reads: False.** Strict blocks both reads and writes of an item written by an uncommitted **other** transaction (a transaction may still touch its own uncommitted data).
10. **ACID consistency $\leftrightarrow$ isolation: False.** Consistency = business-rule preservation. Isolation = no interference between concurrent transactions.

3 Guided Practice (10 Questions)

Attempt each question, read the hint if stuck, then check the full worked solution section before moving to Independent Practice.

3.1 Transaction Basics & ACID (2 questions)

G1 [E] [Transaction Basics] [Concept] A database transaction has a unique TransactionID, begins with an explicit BEGIN statement, performs several read/write operations, and ends with either COMMIT or ROLLBACK. Which ACID property ensures that if the transaction ends with COMMIT, its changes survive a system crash immediately after?

Hint: Think about which ACID property concerns durability and survival after failure.

G2 [M] [ACID] [Scenario] A bank transfer transaction reads Account A's balance (1000), decrements it to 900, reads Account B's balance (500), increments it to 600. After writing the new balance to Account A but before writing to Account B, the system crashes. When recovered, Account A shows 900 but Account B still shows 500. Which ACID property was violated, and why?

Hint: The transaction was not atomic—it was split mid-operation.

3.2 Schedules & Conflicts (2 questions)

G3 [E] [Conflict Identification] [Definition] Do the operations $r_1(X)$ and $r_2(X)$ conflict? Justify your answer.

Hint: Review the conflict definition and the conflict table from Section E.

G4 [M] [Conflict Identification] [Problem] In the schedule $r_1(X) \rightarrow w_1(X) \rightarrow r_2(X) \rightarrow w_2(Y) \rightarrow w_1(Y) \rightarrow c_1 \rightarrow c_2$, identify all pairs of conflicting operations and list the corresponding precedence-graph edges.

Hint: Scan left to right and apply the conflict definition to each operation pair.

3.3 Conflict Serializability (3 questions)

G5 [M] [Precedence Graph] [Problem] Using the schedule from G4, draw the precedence graph. Does a cycle exist?

Hint: Add edges based on the conflicting pairs you found. Check if you can find a path that revisits a node.

G6 [M] [Serializability] [Problem] Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow r_2(Y) \rightarrow w_1(Y) \rightarrow c_1 \rightarrow c_2$. Is this schedule conflict-serializable?

Hint: Build the precedence graph. Topological sort should give the serial order if acyclic.

G7 [H] [Serializability Trap] [Application] A student claims: "If a schedule has only read operations (no writes), it's always serializable." Is this claim true or false? Justify.

Hint: Consider the conflict definition. Can read-only operations create conflicts?

3.4 Recoverability (3 questions)

G8 [E] [Recoverability] [Definition] Define a recoverable schedule in one sentence.

Hint: Focus on the commit order of the reader and the writer.

G9 [M] [Cascadeless] [Scenario] Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow c_1 \rightarrow w_2(X) \rightarrow c_2$. Is this schedule cascadeless? If not, why not?

Hint: Does T_2 read from an uncommitted transaction?

G10 [M] [Strict vs. Cascadeless] [Comparison] Can a schedule be cascadeless but not strict? Provide a short example or explain.

Hint: Cascadeless only restricts reads of uncommitted data. Strict restricts both reads and writes of uncommitted data. Think of a case with a dirty write but no read at all.

4 Guided Practice — Full Worked Solutions

G1 Solution Durability (ACID D). Durability ensures that once a transaction commits (with a successful COMMIT statement), its changes are permanent and survive any subsequent system failures.

G2 Solution Atomicity (ACID A) was violated. The transaction was partially executed: Account A was updated, but Account B was not. Atomicity requires all-or-nothing behavior. The fix is to ensure both writes are written to stable storage before the COMMIT is acknowledged, or neither is written.

G3 Solution No, they do not conflict. The conflict definition requires at least one operation to be a write. Both $r_1(X)$ and $r_2(X)$ are reads, so they can occur in any order without mutual interference.

G4 Solution Scan the schedule $r_1(X) \rightarrow w_1(X) \rightarrow r_2(X) \rightarrow w_2(Y) \rightarrow w_1(Y) \rightarrow c_1 \rightarrow c_2$:

- $r_1(X)$ and $w_1(X)$: same transaction, no edge.
- $r_1(X)$ and $r_2(X)$: both reads, no conflict.
- $w_1(X)$ and $r_2(X)$: different transactions, same item X, write present. Edge: $T_1 \rightarrow T_2$.
- $r_2(X)$ and $w_2(Y)$: different items, no conflict.
- $w_2(Y)$ and $w_1(Y)$: different transactions, same item Y, both writes. Edge: $T_2 \rightarrow T_1$.
- $w_1(Y)$ and c_1 : same transaction, no edge.

Edges: $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$.

G5 Solution Precedence graph: $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$, which forms the cycle $T_1 \rightarrow T_2 \rightarrow T_1$. **Yes, a cycle exists.**

G6 Solution Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow r_2(Y) \rightarrow w_1(Y) \rightarrow c_1 \rightarrow c_2$

Conflicting pairs:

- $w_1(X)$ and $r_2(X)$: different transactions, same item X, write present $\Rightarrow$ conflict. $w_1(X)$ occurs first, so edge $T_1 \rightarrow T_2$.
- $r_2(Y)$ and $w_1(Y)$: different transactions, same item Y, write present $\Rightarrow$ conflict. $r_2(Y)$ occurs **before** $w_1(Y)$, so edge $T_2 \rightarrow T_1$.

Precedence graph: $T_1 \rightarrow T_2 \rightarrow T_1$. **Cycle exists, so not conflict-serializable.**

G7 Solution True. A read-only schedule has no write operations. By the conflict definition, conflicts require at least one write. With no writes, no conflicts exist, so no edges in the precedence graph, so the graph is trivially acyclic, so the schedule is always serializable (in fact, the trivial serial order $(T_1, T_2, \dots)$ works).

G8 Solution A schedule is **recoverable** if whenever T_j reads data written by T_i , the COMMIT of T_i occurs before the COMMIT of T_j .

G9 Solution Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow c_1 \rightarrow w_2(X) \rightarrow c_2$

Is it cascadeless? T_2 reads X, which was written by T_1 . At the time of the read ($r_2(X)$), T_1 has not yet committed (the COMMIT c_1 comes after). **This is not cascadeless.** Cascadeless requires reading only committed data; T_2 read uncommitted data from T_1 .

G10 Solution Yes. Example: $w_1(X) \rightarrow w_2(X) \rightarrow c_1 \rightarrow c_2$.

- **Cascadeless?** Cascadeless only restricts **reads**: a transaction may not read data written by an uncommitted transaction. There are no reads in this schedule at all, so the cascadeless condition is never violated. **Yes, cascadeless.**

- **Strict?** Strict restricts both reads *and writes* by *other* transactions: no other transaction may read *or write* an item written by an uncommitted transaction. Here T_2 writes X while T_1 is still uncommitted ($w_2(X)$ occurs before c_1). This is exactly the kind of dirty write strictness forbids. **Not strict.**

So $w_1(X) \rightarrow w_2(X) \rightarrow c_1 \rightarrow c_2$ is cascadeless but not strict—the write-write case is what separates the two properties, since cascadelessness only ever looks at reads.

5 Independent Practice (18 Questions)

Attempt without referring back. Distribution: 4 ACID/states, 4 schedules/conflicts, 5 conflict serializability, 5 recoverability. Answers appear only in the Answer Key (Section X).

5.1 ACID & Transaction States (4 questions)

I1 [E] [ACID] [Concept] Which ACID property is violated if a transaction partially executes and then fails, leaving the database in an inconsistent state?

1. (A) Atomicity
2. (B) Consistency
3. (C) Isolation
4. (D) Durability

I2 [E] [ACID] [Scenario] A transaction reads data, modifies it locally, and after committing, a power failure occurs 1 second later. The data is lost. Which ACID property was violated?

1. (A) Atomicity
2. (B) Consistency
3. (C) Isolation
4. (D) Durability

I3 [M] [ACID] [Scenario] An invariant rule is that the sum of all account balances must remain constant (money is only transferred, not created/destroyed). A transaction violates this invariant, yet commits successfully. Which property was not enforced?

1. (A) Atomicity
2. (B) Consistency
3. (C) Isolation
4. (D) Durability

I4 [E] [Transaction States] [Definition] A transaction that has finished executing all its operations but has not yet written changes to stable storage is in which state?

1. (A) Active
2. (B) Partially Committed
3. (C) Committed
4. (D) Failed

5.2 Schedules & Conflicts (4 questions)

I5 [E] [Conflict] [Definition] Operations $r_1(Y)$ and $w_2(Y)$ conflict because:

1. (A) They access the same item and belong to different transactions, but neither is a write.
2. (B) They access the same item, belong to different transactions, and at least one is a write.
3. (C) They belong to the same transaction.
4. (D) They access different items.

I6 [E] [Serial vs. Non-Serial] [Definition] A serial schedule is one where:

1. (A) All transactions execute concurrently without any ordering.
2. (B) Transactions execute one after another with no interleaving.
3. (C) The result is equivalent to some serial execution.
4. (D) Only read operations are allowed.

I7 [M] [Conflict Identification] [Problem] Schedule: $w_1(A) \rightarrow r_2(A) \rightarrow w_2(B) \rightarrow r_1(B) \rightarrow c_1 \rightarrow c_2$. How many pairs of conflicting operations are there?

1. (A) 1
2. (B) 2
3. (C) 3
4. (D) 4

I8 [M] [Serial vs. Serializable] [Trap] True or False: “A non-serial schedule cannot be serializable.”

1. (A) True
2. (B) False

5.3 Conflict Serializability (5 questions)

I9 [M] [Precedence Graph] [Problem] Given conflicting pairs: $T_1 \rightarrow T_2$, $T_2 \rightarrow T_3$, $T_3 \rightarrow T_1$. Is the schedule conflict-serializable?

1. (A) Yes, because a topological sort exists.
2. (B) No, because a cycle exists.
3. (C) Cannot be determined.
4. (D) Yes, the serial order is T_1, T_2, T_3 .

I10 [M] [Serializability] [Application] Schedule: $r_1(X) \rightarrow w_1(X) \rightarrow r_2(X) \rightarrow w_2(X) \rightarrow c_1 \rightarrow c_2$. What is the precedence-graph edge?

1. (A) $T_1 \rightarrow T_2$ (from $w_1(X)$ and $r_2(X)$).
2. (B) $T_2 \rightarrow T_1$ (from $w_2(X)$ and $r_1(X)$).

3. (C) Both $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$.
4. (D) No edges (read-only).

I11 [M] [Serializable] [Scenario] A schedule with two transactions has edges $T_1 \rightarrow T_2$ in the precedence graph, with no cycles. Is it serializable?

1. (A) Yes, the serial order is T_1 then T_2 .
2. (B) Yes, the serial order is T_2 then T_1 .
3. (C) No, too few edges.
4. (D) Cannot be determined.

I12 [H] [View Serializability] [Recognition] True or False: “Every conflict-serializable schedule is view-serializable, but the converse does not always hold.”

1. (A) True
2. (B) False

5.4 Recoverability (5 questions)

I13 [E] [Recoverable] [Definition] A schedule is recoverable if:

1. (A) It eliminates all conflicts.
2. (B) Whenever T_j reads data written by T_i , the COMMIT of T_i occurs before the COMMIT of T_j .
3. (C) Only committed data is read.
4. (D) No transaction ever aborts.

I14 [M] [Cascadeless] [Scenario] Schedule: $w_1(X) \rightarrow w_2(X) \rightarrow r_3(X) \rightarrow c_1 \rightarrow c_2 \rightarrow c_3$. Is this cascadeless?

1. (A) Yes, because T_3 reads after T_2 commits.
2. (B) No, because T_3 reads an uncommitted value from T_2 .
3. (C) Cannot be determined without knowing the data values.
4. (D) Yes, because there is only one read.

I15 [M] [Strict] [Definition] A strict schedule requires:

1. (A) No transaction aborts.
2. (B) No transaction reads uncommitted data.
3. (C) No other transaction may read or write data written by an uncommitted transaction.
4. (D) All transactions must commit in order.

I16 [M] [Serializability vs. Recoverability] [Trap] True or False: “If a schedule is serializable, it must also be recoverable.”

1. (A) True
2. (B) False

I17 [H] [Abort/Rollback Propagation] [Scenario] Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow c_1 \rightarrow r_3(X) \rightarrow a_2$ (transaction T_2 aborts after T_1 commits). Which transactions must be rolled back?

1. (A) Only T_2 .
2. (B) T_2 and T_3 .
3. (C) Only T_1 and T_2 .
4. (D) All three.

I18 [H] [Trap] Multiple Choice. Which statement is FALSE?

1. (A) Strict schedules are always cascadeless.
2. (B) Cascadeless schedules are always recoverable.
3. (C) Serializable schedules are always recoverable.
4. (D) A schedule can be recoverable but not serializable.

6 Timed Mini-Test (12 Questions — 18 minutes)

Instructions: Set a timer for 18 minutes. Attempt all 12 without pausing. Answers appear only in the Answer Key.

T1 [E] [ACID] [Concept] Atomicity ensures:

1. (A) A transaction fully completes or is fully rolled back.
2. (B) Concurrent transactions do not interfere.
3. (C) Business rules are preserved.
4. (D) Changes survive crashes.

T2 [E] [Conflict] [Definition] Which pair conflicts?

1. (A) $r_1(X)$ and $r_2(X)$
2. (B) $r_1(X)$ and $w_2(Y)$
3. (C) $r_1(X)$ and $w_2(X)$
4. (D) No pairs in this list conflict.

T3 [M] [Precedence Graph] [Problem] Schedule: $r_1(A) \rightarrow w_2(A) \rightarrow r_2(B) \rightarrow w_1(B) \rightarrow c_1 \rightarrow c_2$. What edges exist?

1. (A) $T_1 \rightarrow T_2$ only.
2. (B) $T_2 \rightarrow T_1$ only.
3. (C) Both $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ (a cycle).
4. (D) No edges.

T4 [M] [Serializability] [Problem] If the precedence graph is acyclic, the schedule is:

1. (A) Serializable.
2. (B) Not serializable.
3. (C) Recoverable.
4. (D) Strict.

T5 [M] [Cycle Trap] [Definition] A cycle in the precedence graph means:

1. (A) The schedule is serializable.
2. (B) The schedule is not conflict-serializable.
3. (C) The schedule is not recoverable.
4. (D) All transactions must abort.

T6 [E] [Recoverable] [Definition] A recoverable schedule ensures:

1. (A) No conflicts exist.
2. (B) If T_j reads from T_i , then T_i commits before T_j commits.

3. (C) Transactions run serially.
4. (D) No aborts ever occur.

T7 [M] [Cascadeless] [Scenario] Schedule: $w_1(X) \rightarrow r_2(X) \rightarrow a_1$. T_1 aborts after T_2 has already read X (written by uncommitted T_1). Under cascading rollback, what is the consequence for T_2 ?

1. (A) T_2 must also be rolled back.
2. (B) T_2 is guaranteed to commit successfully.
3. (C) T_2 is unaffected, since it only performed a read.
4. (D) T_1 is not allowed to abort at this point.

T8 [M] [Strict] [Definition] A strict schedule requires that:

1. (A) Other transactions may read/write an item written by an uncommitted transaction.
2. (B) Other transactions may only read an item after its writer commits/aborts, but may overwrite it before then.
3. (C) No other transaction may read or write an item written by an uncommitted transaction.
4. (D) All transactions must execute serially.

T9 [M] [Hierarchy] [Trap] True or False: “Cascadeless $\implies$ Strict.”

1. (A) True.
2. (B) False.

T10 [H] [Serializability vs. Recoverability] [Trap] Which statement is TRUE?

1. (A) Every serializable schedule is recoverable.
2. (B) Every recoverable schedule is serializable.
3. (C) Serializability and recoverability are independent properties.
4. (D) A schedule cannot be both serializable and recoverable at the same time.

T11 [M] [Serial vs. Serializable] [Concept] A serial schedule is always:

1. (A) Not serializable.
2. (B) Serializable.
3. (C) Requires a precedence graph to verify.
4. (D) Only 2 transactions long.

T12 [H] [Abort Propagation] [Problem] Schedule: $r_2(X) \rightarrow w_1(X) \rightarrow c_1 \rightarrow r_3(X) \rightarrow c_2 \rightarrow a_3$. After T_3 aborts, which other transactions, if any, must be rolled back?

1. (A) Only T_2 and T_1 .
2. (B) Only T_2 .
3. (C) None; T_3 alone.
4. (D) All three.

7 Day 1–3 Cumulative Review (8 Rapid-Fire Questions)

Instructions: These are **rapid-fire** cumulative checks from Days 1–3. Maximum 10 minutes, no reteaching. If you miss one, log it and revisit Days 1–3 materials.

C1 [E] [Day 1] [Keys] A candidate key is best described as:

1. (A) A super key that is minimal.
2. (B) Any key that identifies a row.
3. (C) The key chosen as primary.
4. (D) A foreign key.

C2 [E] [Day 1] [Closure] What is the closure of an attribute set X ?

1. (A) All attributes reachable from X via FDs.
2. (B) All attributes in the relation.
3. (C) All rows with X values.
4. (D) The primary key.

C3 [M] [Day 2] [2NF] 2NF requires:

1. (A) 1NF + no non-prime partial dependency on a composite key.
2. (B) 1NF + no redundancy.
3. (C) No foreign keys.
4. (D) Atomic values.

C4 [M] [Day 2] [3NF] 3NF adds to 2NF the requirement:

1. (A) No transitive dependency of a non-prime on a non-prime via the key.
2. (B) All attributes must be prime.
3. (C) No composite keys.
4. (D) Recovery must be possible.

C5 [E] [Day 3] [BCNF] BCNF differs from 3NF because:

1. (A) BCNF removes the “or A is prime” exception.
2. (B) 3NF is stricter.
3. (C) They are the same.
4. (D) BCNF requires weak entities.

C6 [M] [Day 3] [Lossless] A 2-way decomposition is lossless if:

1. (A) The common attribute is a super key of one side.
2. (B) Both sides have the same number of attributes.

3. (C) No FDs are lost.
4. (D) The decomposition is into BCNF.

C7 [E] [Day 3] [ER Mapping] An M:N relationship between two entities maps to:

1. (A) Foreign key on one side.
2. (B) Separate junction table.
3. (C) Merge into one table.
4. (D) No table needed.

C8 [E] [Day 1] [Lossless/Dependency] Losslessness and dependency preservation are:

1. (A) The same property.
2. (B) Independent properties.
3. (C) Always both true together.
4. (D) Never both true together.

8 Daily Quiz (5 Questions)

Final quick check covering the five core Day 4 concepts. No solutions immediately after; see Answer Key.

Q1 [E] [ACID] [Definition] Which ACID property means a transaction either fully succeeds or is fully rolled back?

1. (A) Atomicity
2. (B) Consistency
3. (C) Isolation
4. (D) Durability

Q2 [M] [Conflict] [Definition] Identify all conflicting pairs in: $r_1(X) \rightarrow w_2(X) \rightarrow r_1(Y)$

1. (A) Only $r_1(X)$ and $w_2(X)$ conflict.
2. (B) Only $w_2(X)$ and $r_1(Y)$ conflict.
3. (C) All three pairs conflict.
4. (D) No pairs conflict.

Q3 [E] [Precedence Graph] [Concept] If a precedence graph has a cycle, the schedule is:

1. (A) Serializable.
2. (B) Not conflict-serializable.
3. (C) Recoverable.
4. (D) Strict.

Q4 [M] [Recoverability] [Definition] A schedule is cascadeless if:

1. (A) Transactions only read committed data.
2. (B) All transactions commit.
3. (C) No conflicts exist.
4. (D) There are no cascading rollbacks.

Q5 [M] [Serializability vs. Recoverability] [Trap] True or False: “Every serializable schedule is recoverable.”

1. (A) True
2. (B) False

9 Complete Answer Key

9.1 Independent Practice Answers

- I1. (A) Atomicity.
- I2. (D) Durability.
- I3. (B) Consistency.
- I4. (B) Partially Committed.
- I5. (B) They access the same item, belong to different transactions, and at least one is a write.
- I6. (B) Transactions execute one after another with no interleaving.
- I7. (B) 2. Pairs: $(w_1(A), r_2(A))$ and $(w_2(B), r_1(B))$.
- I8. (B) False. A non-serial schedule can be serializable.
- I9. (B) No, because a cycle exists $(T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1)$.
- I10. (A) $T_1 \rightarrow T_2$.
- I11. (A) Yes, the serial order is T_1 then T_2 .
- I12. (A) True. This is the key relationship between conflict- and view-serializability.
- I13. (B) Whenever T_j reads data written by T_i , the COMMIT of T_i occurs before the COMMIT of T_j .
- I14. **(B) No, because T_3 reads an uncommitted value from T_2 .** At the point $r_3(X)$ occurs, T_2 has written X but has not yet committed (c_2 comes later in the schedule). So T_3 reads dirty (uncommitted) data from T_2 , violating cascadelessness.
- I15. (C) No other transaction may read or write data written by an uncommitted transaction.
- I16. (B) False. Serializability and recoverability are independent properties.
- I17. (A) Only T_2 . T_1 has already committed, so it cannot be rolled back. T_3 read X only *after* T_1 committed—so T_3 read committed data from T_1 , not dirty data from T_2 (which never wrote anything). Since nobody read from T_2 , its abort does not cascade to any other transaction; only T_2 itself rolls back.
- I18. (C) is FALSE. A serializable schedule may or may not be recoverable.

9.2 Timed Mini-Test Answers

- T1. (A) A transaction fully completes or is fully rolled back.
- T2. (C) $r_1(X)$ and $w_2(X)$ (different transactions, same item, at least one write).
- T3. (C) Both $T_1 \rightarrow T_2$ (from $r_1(A)$ and $w_2(A)$) and $T_2 \rightarrow T_1$ (from $w_2(B)$ and $r_1(B)$).
- T4. (A) Serializable.
- T5. (B) The schedule is not conflict-serializable.
- T6. (B) If T_j reads from T_i , then T_i commits before T_j commits.
- T7. (A) T_2 must also be rolled back. T_2 read a dirty (uncommitted) value of X from T_1 ; when T_1 aborts, that value never existed, so T_2 's work based on it is invalid and must cascade into a rollback of T_2 as well. This is exactly the scenario cascadeless schedules are designed to prevent.
- T8. **(C) No other transaction may read or write an item written by an uncommitted transaction.** (A) describes a schedule with no isolation at all. (B) describes cascadeless-but-not-strict (reads blocked, but dirty overwrites still allowed) — not strict. (D) describes a serial schedule, a stronger and unrelated notion.
- T9. (B) False. Cascadeless does not imply strict.
- T10. (C) Serializability and recoverability are independent properties. A schedule may satisfy only one of them, both together, or neither—one property never implies or rules out the other.
- T11. (B) Serializable. Every serial schedule is trivially serializable.
- T12. **(C) None; T_3 alone.** T_3 reads X at $r_3(X)$, which occurs *after* T_1 commits (c_1) — so T_3 reads committed data from T_1 , not dirty data. T_3 's only operations in this schedule are $r_3(X)$ and its abort a_3 (it never commits). Since no other transaction ever read from T_3 , its abort

cannot cascade to T_1 or T_2 — only T_3 itself is rolled back.

9.3 Cumulative Review Answers

- C1. (A) A super key that is minimal.
- C2. (A) All attributes reachable from X via FDs.
- C3. (A) 1NF + no non-prime partial dependency on a composite key.
- C4. (A) No transitive dependency of a non-prime on a non-prime via the key.
- C5. (A) BCNF removes the “or A is prime” exception.
- C6. (A) The common attribute is a super key of one side.
- C7. (B) Separate junction table.
- C8. (B) Independent properties.

9.4 Daily Quiz Answers

- Q1. (A) Atomicity.
- Q2. (A) Only $r_1(X)$ and $w_2(X)$ conflict.
- Q3. (B) Not conflict-serializable.
- Q4. (A) Transactions only read committed data.
- Q5. (B) False. They are independent properties.

10 Scoring and Diagnostic Interpretation

Score Band	Interpretation and Next Action
< 60%	Revisit transaction states, ACID, and conflict serializability. Focus on precedence graphs and the cycle test. Redo Guided Practice G4–G6 before Day 5.
60 – 75%	Acceptable. Log weak sub-topics in error log. Review the serializability vs. recoverability distinction.
75 – 85%	Good. Log individual misses, proceed as scheduled.
> 85%	Strong. Proceed to Day 5 with confidence.

Note: This is a personal study diagnostic only, not an official IOCL cutoff or scoring benchmark.

11 Revision Checklist

12-Item Revision Checklist — tick before moving to Day 5

- ☐ I can define a transaction and explain why they exist.
- ☐ I can state all four ACID properties and give a practical example for each.
- ☐ I know transaction states: Active, Partially Committed, Committed, Failed, Aborted, Terminated.
- ☐ I can interpret schedule notation: $r_i(X)$, $w_i(X)$, c_i , a_i .
- ☐ I know the exact rule for conflicting operations (different transactions, same item, at least one write).
- ☐ I can construct a precedence graph from a schedule and identify conflicting pairs.
- ☐ I know: acyclic graph $\implies$ conflict-serializable; cycle $\implies$ NOT conflict-serializable.
- ☐ I can derive a topological sort from an acyclic precedence graph to find a valid serial order.
- ☐ I know view-serializability is weaker than conflict-serializability (every conflict-serializable is view-serializable, but not vice versa).
- ☐ I understand recoverable, cascadeless, and strict schedules and the hierarchy: Strict $\implies$ Cascadeless $\implies$ Recoverable. I know strictness is specifically about *other* transactions touching uncommitted data — a transaction may always read/write its own uncommitted writes.
- ☐ **I know serializability and recoverability are independent properties**—a schedule can be one, both, or neither.
- ☐ I can recognize all 10 common traps and avoid them on the exam.

12 Things I Got Wrong Today

Tag the category before logging specifics:

Question	Topic/Category	My Mistake	Correct Rule

13 One-Page Cheat Sheet

Transaction Basics

ACID: A = all-or-nothing; C = business rules preserved; I = no interference; D = survives crashes.

States: Active $\rightarrow$ Partially Committed $\rightarrow$ Committed. Or: Active $\rightarrow$ Failed $\rightarrow$ Aborted.

Schedule Notation

$r_i(X)$ = T_i reads X. $w_i(X)$ = T_i writes X. c_i = T_i commits. a_i = T_i aborts.

Serial: No interleaving. **Serializable:** Equivalent to some serial execution.

Conflicts

Different transactions, same item, at least one write = conflict.

$r_i(X)$ and $r_j(X)$ do **NOT** conflict.

Precedence Graph

1. Identify all conflicting pairs.
2. Draw $T_i \rightarrow T_j$ when T_i 's operation precedes T_j 's conflicting operation.
3. Acyclic $\implies$ conflict-serializable. Cycle $\implies$ NOT.

Serializability vs. Recoverability

Independent properties. A schedule can be:

- Serializable but non-recoverable.
- Recoverable but non-serializable.
- Both (ideal).
- Neither (bad).

Recoverability Hierarchy

Strict $\implies$ Cascadeless $\implies$ Recoverable.

Recoverable: Reader commits after writer.

Cascadeless: No reading of uncommitted data (from another transaction).

Strict: No *other* transaction may read or write data written by an uncommitted transaction (a transaction may still touch its own uncommitted writes).

High-Yield Traps (Review Daily)

1. $r_i(X)$ and $r_j(X)$ do NOT conflict.
2. Reverse precedence-graph edge direction = trap.
3. Cycle $\neq$ serializable.
4. Serial $\neq$ serializable (serializability allows interleaving).

5. Conflict-serializable $\subset$ view-serializable (not the reverse).
6. Serializability $\not\Rightarrow$ recoverability.
7. Recoverability $\not\Rightarrow$ serializability.
8. Cascadeless $\neq$ strict.
9. Strict blocks reads AND writes of uncommitted data *by other transactions*.
10. Consistency (ACID C) $\neq$ Isolation (ACID I).

DAY 4 — SUPPLEMENT

Recovery & Distributed Transactions

6. Crash Recovery & Recovery Protocols

6.1 Why Recovery Matters

When a transaction commits, a DBMS guarantees durability: the data will persist even if the system crashes. This happens via logging and recovery protocols. The two main approaches:

6.2 Write-Ahead Logging (WAL)

- **The Protocol:** Before any data page is written to disk, its log entry must first be written (forced) to the log file.
- **Log Format:** [TransactionID, Operation, OldValue, NewValue, LSN (Log Sequence Number)]
- **Key Principle:** Log $\Rightarrow$ Disk BEFORE Data $\Rightarrow$ Disk (guarantees we can undo/redo)

6.3 Undo/Redo Logging

Two strategies depending on when pages are written:

Undo-Only Logging: Write log before writing data. Allows only UNDO (rollback uncommitted changes).

Redo-Only Logging: Write log after changes are in memory. Allows only REDO (reapply committed changes).

Undo/Redo Logging: Writes both BEFORE and AFTER images. Most flexible (supports both UNDO and REDO).

6.4 Checkpoints

- A **checkpoint** is a snapshot: at time t , we log all dirty pages to disk and mark a checkpoint record.
- Recovery process: scan log backwards from crash point to last checkpoint, then determine what to UNDO/REDO.
- **Benefit:** Reduces recovery time; no need to replay the entire log from the beginning.

6.5 ARIES (Algorithms for Recovery and Isolation Exploiting Semantics)

Industry-standard recovery protocol (used in DB2, SQLServer, PostgreSQL variants). Three phases:

Phase 1 — Analysis: Scan log from checkpoint to crash. Identify dirty pages and active transactions.

Phase 2 — Redo: Replay all changes (committed and uncommitted) from the redo log forward.

6.5.1 ARIES — Going One Level Deeper

Addendum inserted for extra depth beyond the Day 4 recognition-level summary.

The three-phase summary above (Analysis → Redo → Undo) is correct and is all a CBT MCQ will test. Two mechanics worth being able to explain out loud if an interview round follows the CBT:

Why redo runs before undo, and why it redoes aborted transactions too

ARIES does not try to distinguish committed from uncommitted work during the Redo phase — it simply replays the physical log forward to reconstruct the exact database state that existed at the moment of the crash, no more and no less. This is called **“repeating history.”** Only after that exact pre-crash state is rebuilt does the Undo phase look at which transactions never committed and reverses just their changes. Trying to skip redoing a transaction that “will end up aborting” would require knowing the outcome before finishing the log scan, which the log doesn’t cleanly give you mid-pass — so ARIES redoes everything uniformly first, then corrects afterward.

Compensation Log Records (CLRs)

When the Undo phase reverses an update, that reversal is itself written to the log as a **Compensation Log Record**. This is what makes ARIES resilient to a second crash during recovery: if the system fails again while undoing, the next recovery pass sees the CLRs already written and knows not to undo the same action twice. Without CLRs, a crash-during-recovery could re-undo an already-undone write and corrupt data.

INTERVIEW INSIGHT

One-line answer if asked to summarize ARIES beyond the three phases: “Redo repeats history unconditionally using the log, then undo rolls back only the losers — and every undo step is itself logged as a CLR, so recovery is safe even if it gets interrupted and has to restart.”

Phase 3 — Undo: Scan log backwards, undo all uncommitted transactions (rollback).

Key Insight: ARIES separates redo (restores consistency) from undo (restores isolation). This allows committed data to survive and uncommitted changes to be cleanly reversed, even after a cascade of failures.

7. Distributed Transactions

7.1 The Problem

ACID guarantees break when data spans multiple database servers or multiple nodes. A commit at Node A might succeed while Node B fails. How do we maintain all-or-nothing semantics across nodes?

7.2 Two-Phase Commit (2PC)

A coordination protocol between a **Coordinator** (typically the client) and multiple **Participants** (database nodes):

Phase 1 — Prepare/Voting:

- Coordinator sends "prepare to commit" to all participants.
- Each participant locks resources, runs the transaction locally, and votes YES (ready to commit) or NO (abort).
- Participants wait for coordinator's decision (cannot commit/abort yet).

Phase 2 — Commit/Abort:

- If all vote YES, coordinator sends COMMIT; all participants commit and release locks.
- If any vote NO (or coordinator timeout), coordinator sends ABORT; all participants rollback.

Guarantee: Atomic commitment across all nodes. Either all commit or all abort; no partial commits.

7.3 Limitations of 2PC

- **Blocking:** Participants must hold locks during Phase 1; if coordinator fails, they wait indefinitely.
- **Latency:** Two round trips of network communication (slow across WANs).
- **Scalability:** Does not scale to many nodes or across geographic regions.

7.4 The CAP Theorem (Brief Intro)

When building distributed systems, you can guarantee at most **two of three** properties:

Consistency: All nodes see the same data at the same time.

Availability: The system responds to all requests (even if some nodes are down).

Partition Tolerance: The system continues operating when the network is split (nodes cannot communicate).

Implication for ACID: 2PC guarantees Consistency but sacrifices Availability (blocks) and Partition Tolerance (if coordinator is down, nothing happens). Distributed databases like Cassandra, DynamoDB choose Availability + Partition Tolerance, accepting weaker consistency (eventual consistency).